

STREAMINGGS: Voxel-Based Streaming 3D Gaussian Splatting with Memory Optimization and Architectural Support

Chenqi Zhang^{1*}, Yu Feng^{1*}, Jieru Zhao^{1†}, Guangda Liu¹, Wenchao Ding², Chentao Wu¹, Minyi Guo¹

¹ School of Computer Science, Shanghai Jiao Tong University, ² Academy for Engineering & Technology, Fudan University
{zhangchenqi123, y-feng, zhao-jieru}@sjtu.edu.cn *Equal contribution. †Corresponding author.

Abstract—3D Gaussian Splatting (3DGS) has gained popularity for its efficiency and sparse Gaussian-based representation. However, 3DGS struggles to meet the real-time requirement of 90 frames per second (FPS) on resource-constrained mobile devices, achieving only 2 to 9 FPS. Existing accelerators focus on compute efficiency but overlook memory efficiency, leading to redundant DRAM traffic. We introduce STREAMINGGS, a fully streaming 3DGS algorithm-architecture co-design that achieves fine-grained pipelining and reduces DRAM traffic by transforming from a tile-centric rendering to a memory-centric rendering. Results show that our design achieves up to $45.7\times$ speedup and $62.9\times$ energy savings over mobile Ampere GPUs.

Index Terms—3D gaussian splatting, rendering, accelerator

I. INTRODUCTION

In virtual and augmented reality (VR/AR), photo-realistic and real-time rendering is essential for delivering immersive user experiences. Among various rendering techniques [1]–[6], 3D Gaussian Splatting (3DGS) [7]–[12] has quickly gained its popularity for its efficient rendering process and sparse Gaussian representation (also called Gaussian ellipsoids) of scene geometries.

Nevertheless, 3DGS still falls short of the real-time requirement in the VR/AR domain, namely 90 frames per second (FPS) [13]–[15], on mobile devices like VR headsets. Our experiments show that 3DGS merely achieves 2 to 9 FPS on the Nvidia Orin NX [16]¹. Although several studies have proposed dedicated accelerators for 3DGS [18], their designs primarily optimize compute efficiency and often overlook off-chip memory traffic during rendering, making them hard to satisfy the real-time requirement in real-world scenarios.

Algorithmically, existing methods [7], [18] focus on a tile-centric rendering paradigm which requires excessive intermediate data communication between stages, leading to frequent off-chip traffic in Fig. 1a. Our characterization shows that, under a 90 FPS real-time requirement, the DRAM traffic of the current rendering paradigm would exceed the bandwidth limits of today’s mobile devices on real-world scenes (Sec. II-B).

¹The performance of the Ampere GPU (3.7 TFLOPS, released in February 2023) is comparable to that of the GPU (3.5 TFLOPS) in the latest Snapdragon XR2 [17], released in September 2023 for VR.

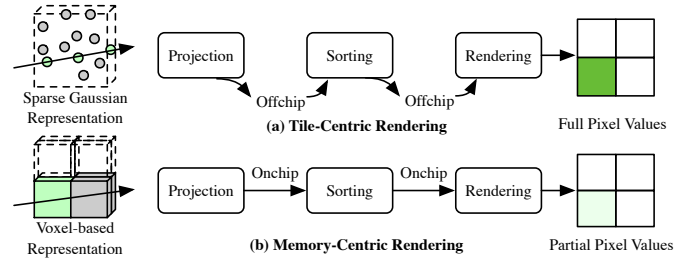


Fig. 1. Tile-centric rendering (a) vs. memory-centric rendering (b). The computation pattern is highlighted in green. Tile-centric rendering computes the complete pixel values tile-by-tile. Memory-centric rendering computes the partial pixel values voxel-by-voxel and regularizes the memory accesses.

The high data communication in the conventional tile-centric rendering comes from sequential processing ray-Gaussian intersections for each individual pixel, leading to redundant off-chip traffic and memory irregularity [18]–[21]. To address the bandwidth limitation, we argue to shift the rendering paradigm from a *tile-centric* to a *memory-centric* approach. Here, we introduce a fully streaming algorithm for 3DGS to address memory access irregularity (Sec. III). Our key idea is to regularize sparse Gaussian representations by partitioning the scene into voxels and performing on a voxel basis as shown in Fig. 1b. With all the points within a voxel stored contiguously in DRAM, we allow all points within a voxel to be streamed onto the on-chip buffer and only write the final pixel values off-chip, completely eliminating the intermediate off-chip traffic between stages.

While our memory-centric approach eliminates the intermediate off-chip traffic, our approach potentially introduces unnecessary off-chip traffic at the voxel streaming, as all points within a voxel are loaded on-chip as a whole. To reduce the unnecessary off-chip traffic, we propose hierarchical filtering to progressively exclude irrelevant Gaussians (Sec. III-B).

Specifically, we split each Gaussian feature into two halves and intentionally keep the first half lightweight. We then perform coarse yet lightweight computation to filter out irrelevant points based on the first half (i.e., the position and radius). Next, we perform precise filtering to further remove the irrelevant Gaussians by loading the second half. To further

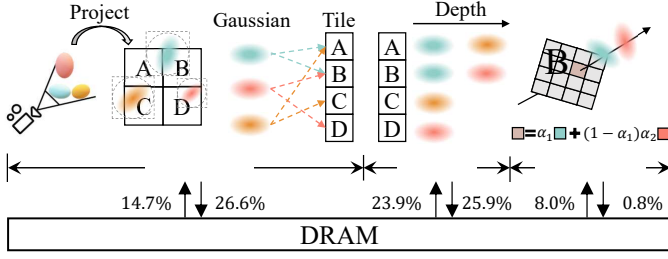


Fig. 2. The overall rendering pipeline of 3DGS. The percentage numbers represent the DRAM traffic proportion across stages.

reduce the overhead of feature loading, we compress the second half using vector quantization. By storing a compact codebook on-chip, we just need to fetch their lightweight indices from DRAM. In this way, we drastically reduce the DRAM traffic at the voxel streaming.

To unleash the full potential of our fully streaming algorithm, we co-design an efficient accelerator (Sec. IV), including a voxel sorting unit to accelerate the determination of voxel rendering order and a dedicated hierarchical filtering unit to support Gaussian filtering at the voxel streaming. Coupled with our algorithmic optimizations, our design achieves a $45.7\times$ speedup and $62.9\times$ energy savings compared to a Nvidia mobile Ampere GPU [16]. Even compared against the state-of-the-art accelerator, GScore [18], we can achieve a $2.1\times$ speedup and $2.3\times$ energy savings.

In summary, our contributions are as follows:

- We introduce a *memory-centric* rendering paradigm that can achieve fully streaming 3DGS rendering, completely avoiding intermediate off-chip traffic.
- We propose a hierarchical filtering scheme to further reduce the data traffic of the voxel streaming.
- With our architectural supports, our optimizations achieve an order of magnitude speedup and energy savings compared to a mobile Ampere GPU. We also achieve a $2.1\times$ speedup and $2.3\times$ energy savings against GScore.

II. BACKGROUND AND CHARACTERIZATION

A. Background

Overall, 3DGS uses Gaussian ellipsoids to render a scene in a tile-by-tile fashion. Fig. 2 shows that this process can be classified into three stages:

- **Projection.** This stage projects each Gaussian ellipsoid onto the 2D screen and determines its intersected tiles as shown on the left side of Fig. 2. Each Gaussian might intersect with multiple tiles, as the colored dashed arrow lines show. Meanwhile, this stage calculates Gaussian properties, such as color and depth, for the later stages.
- **Sorting.** Each tile then sorts its intersected Gaussians based on Gaussian depths to ensure the correct rendering order from the closest to the farthest.
- **Rendering.** Every pixel within a tile traverses the same sorted Gaussian list and accumulates Gaussian colors to itself via alpha blending. For instance, we show tile B needs to blend two Gaussians, a blue one and a red one.

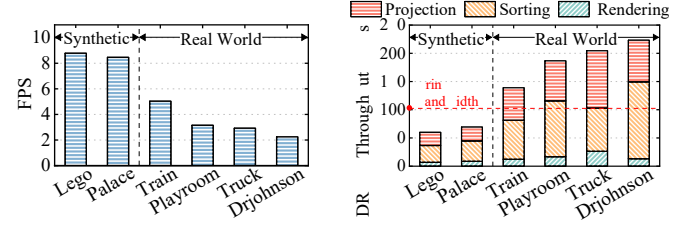


Fig. 3. FPS of different scenes on a mobile SoC, Orin NX. 3DGS cannot achieve real-time on both synthetic and real-world scenes.

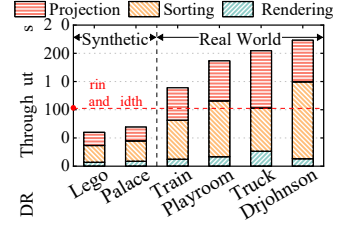


Fig. 4. The DRAM bandwidth requirement for 90 FPS under different scenes. The red dashed line highlights the bandwidth limit of Orin NX.

B. Characterization

Performance. We first show that 3DGS algorithms cannot achieve real-time performance on mobile devices. Fig. 3 characterizes the FPS of 3DGS [7] across various scenes on NVIDIA’s Jetson Orin NX [16]. As the rendering scenes become more complex, shifting from synthetic to real-world scenes, performance drops from 8.5 to 4.9 FPS.

DRAM Traffic. We then show that DRAM traffic has a strong correlation with performance. Fig. 4 illustrates the DRAM throughput required to achieve the real-time requirement (90 FPS) for AR/VR applications [13], [14] across different rendering scenes. For real-world scenes, the DRAM bandwidth demand exceeds the bandwidth limit of a recently released mobile device, Nvidia Orin NX [16], as highlighted in the red dashed line. This means that the data communication requirement alone has made it impossible to achieve real-time.

We further dissect the DRAM traffic contributions across different scenes. As shown in Fig. 4, *projection* and *sorting* dominate the overall off-chip data communication, together accounting for 90% of the total DRAM traffic. The reasons that *projection* and *sorting* become the major contributors are:

- In projection, each Gaussian requires loading 59 parameters into the on-chip buffer and writing the processed features and intersection metadata back to DRAM, accounting for 41% of DRAM traffic.
- In sorting, repetitive read and write operations for progressive Gaussian sorting lead to repetitive DRAM access, taking up over 49% of DRAM traffic.

Overall, the intermediate data communication accounts for 85% of the total off-chip traffic in Fig. 2. This result motivates us to propose a fully-streaming algorithm that minimizes the off-chip traffic. Sec. III presents our algorithm design and Sec. IV explains the architectural support for our algorithm.

III. ALGORITHM DESIGN

Sec. III-A first gives an algorithm overview and challenges that we need to address. Then, Sec. III-B and Sec. III-C explain the key components in our algorithm and the data layout that supports our algorithm, respectively.

A. Overview

The main downside of the original 3DGS pipeline is that it renders a frame tile-by-tile and each tile executes three stages sequentially. The reason is that the intermediate data of each

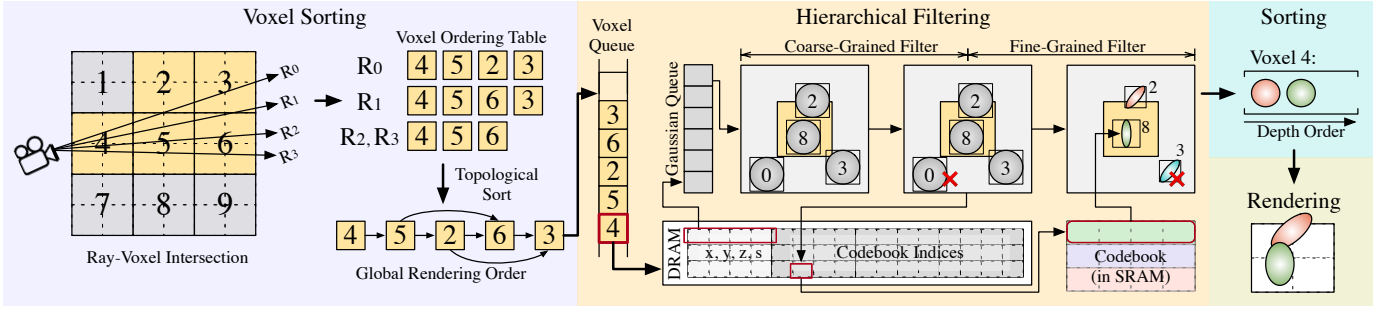


Fig. 5. The overview of our fully-streaming algorithm. Our algorithm initially partitions the entire 3D scene into small voxels. For each image tile, we first establish a rendering order of the intersected voxels via topological sort, then, process these voxels sequentially. During each voxel processing, we apply hierarchical filtering to further reduce the Gaussian loading within a voxel before the remaining sorting and rendering stages. The final pixel values are obtained by accumulating all the partial results across voxels.

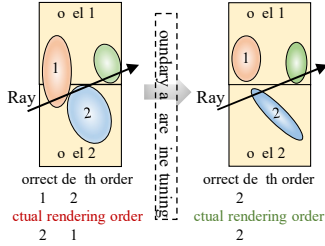


Fig. 6. An example of incorrect rendering order among Gaussians and the effect of boundary-aware fine-tuning.

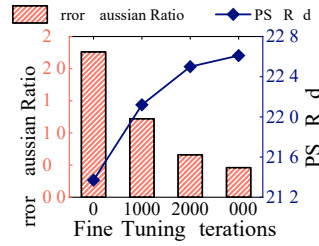


Fig. 7. Changes in the ratio of Gaussians with incorrect depth order and PSNR during the boundary-aware fine-tuning process.

stage is too large to be accommodated on-chip, inevitably, introducing frequent off-chip traffic during the execution.

Idea. To eliminate frequent off-chip communications at intermediate stages, our idea is to partition an entire 3D scene into small voxels and incrementally render the entire scene voxel-by-voxel. By processing one voxel at a time, our algorithm guarantees the intermediate data between stages can be completely held on-chip, thus, no off-chip traffic occurs.

Fig. 5 gives an overview of our algorithm. Our algorithm renders a group of pixels together, e.g., R_0 to R_3 are in the same group. For each group, we first identify which voxels intersect with these pixel rays and store the results in the voxel rendering table. Once each pixel's intersected voxels are determined, we establish their global rendering order. Then, each voxel goes through hierarchical filtering, sorting, and rendering in order, and obtain the partial pixel values of this tile. The final pixel values are obtained by accumulating all the partial results across voxels. By rendering one voxel at a time, each stage requires minimal data communication, thus, no off-chip traffic is required throughout the execution.

Challenges. However, there are three challenges when applying our algorithm:

- **Inter-Voxel Order.** Pixels within a single group often intersect with multiple voxels, as shown in Fig. 5. The rendering order for each pixel should be from the closest voxel to the farthest. The first challenge is to ensure the correct global order for all intersected voxels.
- **Cross-Voxel Gaussian Order.** Recall, each pixel is re-

quired to render Gaussians from the closest to the farthest in 3DGS, but the correct voxel-level ordering does not always guarantee the rendering order correctness at Gaussian-level granularity. For instance, on the left side of Fig. 6, Gaussian 2 in voxel 2 renders before Gaussian 1 in voxel 1, whereas the correct order should be reversed.

- **Redundant Gaussians in Voxels.** Lastly, each voxel includes Gaussians within close proximity; however, loading an entire voxel might unintentionally bring unrelated Gaussians on-chip. Leaving these redundant Gaussian without care would introduce excessive DRAM traffic.

B. Fully Streaming Algorithm

In this section, we explain how we address these challenges.

Inter-Voxel Order. To establish the correct voxel order across pixels in each tile, we first collect the rendering order of individual pixels in a voxel ordering table in Fig. 5. Based on individual rendering orders, we construct a direct acyclic graph (DAG), where each voxel is represented as a node, and edges represent the rendering dependencies. We then apply a topological sort [22] on the DAG to guarantee that no cycles in this graph and all the render dependencies are satisfied. The sorted order is then used in the remaining process.

Cross-Voxel Gaussian Order. To ensure correct rendering order among Gaussians, we observe that the incorrect order occurs only when a Gaussian spans across multiple voxels. In Fig. 6, Gaussians should be rendered from Gaussian 1 (the closest) to Gaussian 3 (the farthest). However, with voxel-by-voxel rendering, Gaussian 2 is incorrectly rendered before Gaussian 1. Fig. 7 shows that incorrect ordering can lead to a degradation in rendering quality.

To alleviate this issue, we introduce a *boundary-aware fine-tuning* to adjust cross-voxel Gaussians. Specifically, we keep each Gaussian position fixed to retain the scene geometry while fine-tuning its remaining trainable parameters, such as scale, orientation, etc. The fine-tuning reduces the size and modifies the orientation of Gaussians, minimizing their likelihood of spanning across other voxels.

In our fine-tuning, we propose a new loss function to punish the cross-boundary Gaussians,

$$\mathcal{L} = \mathcal{L}_{origin} + \beta \mathcal{L}_{CBP}, \quad (1)$$

where $\mathcal{L}_{origin}$ is the original loss in 3DGS and $\mathcal{L}_{CBP}$ is the cross-boundary penalty loss which penalizes Gaussians that span across voxel boundaries. β is a hyper-parameter to balance the influence of $\mathcal{L}_{CBP}$.

$\mathcal{L}_{CBP}$ minimizes the scale of the Gaussians that cross voxel boundaries and is expressed as,

$$\mathcal{L}_{CBP} = \frac{1}{N} \sum_{i=0}^{N-1} S_i T_i, \quad (2)$$

$$\text{where } T_i = \begin{cases} 1, & \text{if } \text{depth}_i < \max_{j \in [0, i)} \{\text{depth}_j\}, \\ 0, & \text{otherwise,} \end{cases}$$

where N is the total number of Gaussians, S_i is the maximum scale of Gaussian i . T_i is an indicator that checks if current Gaussian i has the largest depth compared to all previously rendered Gaussians. If this condition is not satisfied, we penalize it by setting T_i to be 1.

Fig. 7 illustrates the effectiveness of our boundary-aware fine-tuning. After 3,000 iterations, the percentage of cross-boundary Gaussians decreases from 2.3% to 0.4%, while the rendering quality regains from 21.37 dB to 22.61 dB.

Redundant Gaussians in Voxels. Recall that during projection, each Gaussian requires 59 parameters. To avoid irrelevant Gaussians, we propose a two-phase *hierarchical filtering* to progressively fetch Gaussian parameters in Fig. 5.

The first phase serves as a lightweight but coarse-grained filter that approximates whether Gaussians are intersected with the image tile. In this phase, each Gaussian only requests 4 parameters (3D coordinates and maximum scale) from off-chip memory to compute the projected center and the maximum projected radius with respect to the image tile. If a Gaussian does not pass the coarse-grained filter, we skip its remaining computations and avoid loading its remaining 55 parameters. For example, Gaussian 0 in Fig. 5 does not intersect with the image tile in yellow, we can safely remove it from the subsequent computation.

In the second fine-grained filtering, we identify the actual Gaussians that are intersected with the image tile by computing precise projection. For example, after getting shapes and orientations, we can know that Gaussian 3 in Fig. 5 does not intersect with the image tile actually and hence can skip its subsequent sorting and rendering. To further reduce the off-chip traffic, we propose a novel data compression scheme to encode the remaining parameters into a compact codebook. This way, we only need to load codebook indices from DRAM instead of the original parameters. We explain our data compression scheme in Sec. III-C. Combining coarse- and fine-grained filtering, our hierarchical filtering reduces the number of Gaussians processed in each voxel by 76.3%.

C. Customized Data Layout

To reduce data communication in hierarchical filtering, we propose a customized data layout in Fig. 8. Overall, we divide all Gaussian features into two halves, which are stored separately in DRAM and used at different filtering phases. The first half, containing 4 parameters (the 3D coordinates

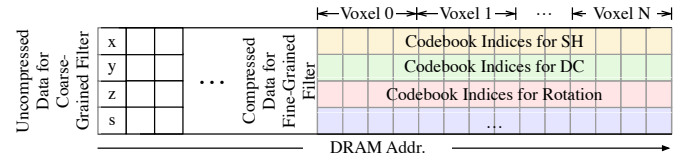


Fig. 8. The DRAM Data layout. We separate the Gaussian features into two halves for two filtering phases and compress the data for the second phase using vector quantization to reduce the off-chip communication.

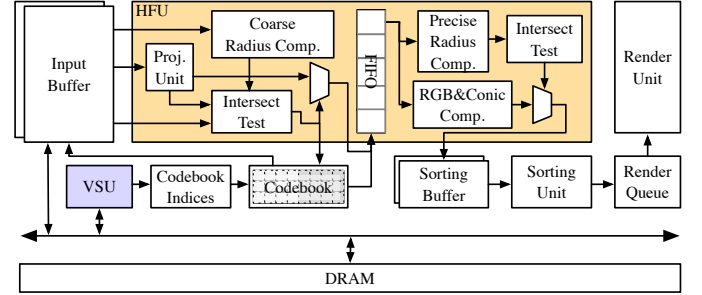


Fig. 9. Overview of STREAMINGGS accelerator, consisting of four components: a voxel sorting unit (VSU), a hierarchical filtering unit (HFU), a sorting unit, and a rendering unit. Our contributions are highlighted in colors: HFU is in yellow and VSU is in purple. The detailed design of VSU is in Fig. 10.

and maximum scale), is used for the coarse-grained filter. The second half, used in the fine-grained filter, includes the remaining parameters, such as rotation matrices.

To alleviate DRAM traffic, we apply the vector quantization to compress the second half, so that we store only codebook indices off-chip while keeping the codebook itself on-chip for decoding. In addition, we encode different parameters into separate codebooks to preserve quantization precision and adopt the quantization-aware fine-tuning [9] to update the Gaussian parameters, allowing the indices to capture feature variations without loss of detail. Our result shows that, during the voxel streaming, only the codebook indices are fetched from DRAM and reduce 92.3% DRAM traffic. Note that, we only compress the second half, because we empirically find that compressing the first half leads to accuracy loss.

IV. ARCHITECTURAL SUPPORT

A. Overview

To support our fully-streaming algorithm, we propose our co-designed accelerator in Fig. 9. Our architecture consists of four components: a voxel sorting unit (VSU), a hierarchical filtering unit (HFU), a sorting unit, and a rendering unit.

Given a set of ray directions, VSU first identifies and sorts the intersected voxels. Then, for each intersected voxel, HFU applies a hierarchical filter to identify intersected Gaussians. Both designs will be explained in Sec. IV-B and Sec. IV-C, respectively. Once HFU obtains the filtered Gaussians, sorting and rendering units execute the sorting and rendering stage from their individual buffer, respectively. Both sorting and rendering designs are adopted from GScore [18]. However, we simplify the sorting unit by just adopting the bitonic sorting unit from GScore, as our voxel-based rendering only requires establishing the rendering order for Gaussians within a voxel.

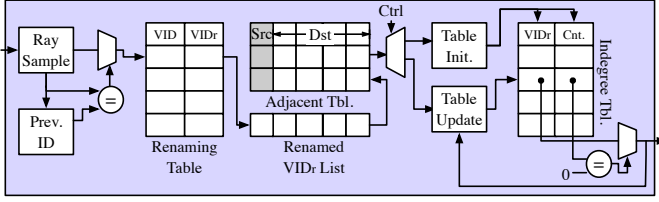


Fig. 10. The design of voxel sorting unit (VSU). It first identifies the intersected voxels for every pixel within a tile and then establishes the global rendering order of these voxels for sorting and rendering.

B. Voxel Sorting Unit (VSU)

The architecture of VSU is illustrated in Fig. 10. For pixels within a group, VSU samples along each pixel ray to identify the intersected voxels and performs a topological sort to establish the global rendering order of these voxels. Since we partition the entire scene into voxels offline, the voxel ID (VID) can be directly computed by the coordinate of a ray sample. To reduce the VID range, we further filter out those empty voxels with no Gaussians by augmenting a renaming table in VSU to reassign each VID with a new VID_r.

Each time we obtain a renamed VID_r list for one pixel, we use this list to construct an adjacent table. This adjacent table functions like a cache to help the dependency lookup among voxels. Each table entry has a tag and a value, representing a source voxel and all its corresponding destination voxels, respectively. The adjacent table is constructed once VSU finishes iterating all pixel rays in a group.

VSU then uses the adjacent table to initialize an in-degree table. Each entry in the in-degree table is directly indexed by a VID_r value and stores its in-degree count, which records the number of source voxels this VID_r has. If an in-degree of a VID_r becomes zero, this VID_r has no source dependency and can be written to the later stage. Each time VSU outputs a VID_r, VSU also updates the in-degree table accordingly. The topological sort finishes when the in-degree table is empty.

C. Hierarchical Filtering Unit (HFU)

Once VSU generates the rendering order of intersected voxels, HFU starts to load one voxel at a time and process the Gaussians within this voxel. Overall, HFU, highlighted in yellow in Fig. 9, consists of a frontend and a backend, responsible for coarse-grained and fine-grained filtering, respectively.

Coarse-Grained Filter. HFU only reads each Gaussian coordinate and maximum scale in the coarse-grained filter, computing its projected position and coarse radius. These results are sent to the intersection test unit to determine if the Gaussian intersects with the current image tile. If the condition is met, HFU signals the codebook to decode the remaining parameters from the codebook buffer, as shown in Fig. 9. The decoded parameters, along with the uncompressed parameters, are then fed into a FIFO for fine-grained filtering.

Fine-Grained Filter. This step computes the exact radius computation and performs the intersection test. Meanwhile, the fine-grained filter unit also computes the RGB values and conic matrix from decoded parameters, such as scales and rotation. Based on the intersection result, HFU conditionally

TABLE I
CONFIGURATION AND AREA

Unit	Configuration	Area [mm ²]
Voxel Sorting Unit	1 Unit	0.06
Hierarchical Filtering Unit	4 Units	0.79
Sorting Unit	2 Units	0.04
Rendering Unit	64 Units	2.53
SRAM (Input Buffer, Codebook, others)	355KB	1.95
Total		5.37

outputs a Gaussian RGB and conic results to the sorting buffer if this Gaussian passes the intersection test.

In our design, the coarse-grained filter largely reduces the computation (from 427 MACs to 55), compared to fine-grained filters. In addition, the coarse-grained filter avoids decoding and reading unnecessary Gaussians to FIFO in HFU.

V. EVALUATION

A. Experimental Setup

Hardware Setup. Our accelerator consists of 1 VSU, 4 HFUs, 2 sorting units, and $4 \times 4 \times 4$ rendering units at 1GHz clock frequency. Each HFU has 4 coarse-grained filter units (CFUs) and 1 fine-grained filter unit (FFU). The sorting unit and the rendering unit are identical to the bitonic sorting unit and the volume rendering unit in GScore, respectively [18]. The overall on-chip buffer consists of a double-buffered input buffer with a size of 16 KB, a 250 KB codebook buffer, and the remaining 89 KB to store intermediate data between stages.

Our architectural design is developed using an EDA process and synthesized with Synopsys and Cadence tools on TSMC's 32 nm FinFET technology. SRAM components are estimated by CACTI 7.0 [23] at 32 nm technology. The DRAM model in our simulations is based on Micron's 16 Gb LPDDR3 with 4 channels according to its datasheet [24], with energy consumption data sourced from Micron System Power Calculators [25].

Software Setup. We configure a codebook with 4096 entries for scale, rotation, and DC, and a codebook with 512 entries for SH coefficients in Sec. III-C. The voxel size is set to 2 for real world scenes and 0.4 for synthetic scenes. The hyper-parameter, β , in Sec. III-B is set to 0.05.

Area. STREAMINGGS design has a total of 5.37 mm² in Tbl. I. The major area contributors are 4 HFUs (0.79 mm²), 64 rendering units (2.53 mm²), and on-chip buffers (1.95 mm²). Overall, our design has a similar area compared to GScore (5.53 mm²) which is scaled to 32 nm by DeepScaleTool [26].

Workloads. We evaluate both synthetic and real-world datasets: Synthetic-NSVF [27], Synthetic-NeRF [3], Tanks&Temples [28] and Deep Blending [29]. Each scene is initially trained with a default 3×10^4 iterations, followed by 3×10^3 iterations of boundary-aware fine-tuning (Sec. III-B) and 5×10^3 iterations of quantization-aware fine-tuning (Sec. III-C). We evaluated our method on three widely-adopted 3DGS algorithms, including original 3DGS [7], Mini-Splatting [8], and LightGaussian [10].

Baselines. We compare against two hardware baselines: Nvidia GPU Orin NX [16] and a dedicated accelerator, GScore [18]. GPU performance and power consumption are

TABLE II

QUANTITATIVE COMPARISON OF RENDERING QUALITY (PSNR $\uparrow$) ACROSS DIFFERENT DATASETS AND ALGORITHMS.

Dataset \ Algorithm	Train	Truck	Playroom	Drjohnson	Lego	Palace
3DGS	22.54	26.65	30.18	29.21	36.11	38.56
Ours	22.52	26.61	30.27	29.07	36.02	38.52
Mini-Splatting	21.49	25.19	30.32	29.23	36.20	39.00
Ours	21.44	25.11	30.37	29.34	36.18	38.98
LightGaussian	22.29	26.02	28.58	25.87	35.18	37.76
Ours	22.32	25.89	28.47	25.79	35.15	37.68

obtained using built-in hardware measurements. We implement GScore based on its published specifications and validate our results against reported data.

Variants. We evaluate three variants:

- w/o VQ+CGF performs our fully-streaming algorithm without vector quantization and coarse-grained filtering.
- w/o CGF performs our fully-streaming algorithm without coarse-grained filtering.
- STREAMINGGS is our full-fledged version including all optimization techniques.

B. Rendering Quality

Tbl. II shows a quantitative comparison of rendering quality between the original pipeline and our fully streaming pipeline across three 3DGS algorithms. Overall, our fully streaming pipeline shows no accuracy degradation in visual quality. On average, our pipeline drops the rendering quality by 0.04 dB. Our results sometimes achieve higher PSNR than the original approach. Our rendering results are also presented here: [link](#).

C. Performance and Energy Efficiency

Fig. 11 illustrates the performance and energy savings of our design against three hardware baselines. All values are normalized against the corresponding Orin NX. Both speedup and energy numbers are averaged across four datasets.

On average, STREAMINGGS achieves $45.7\times$ speedup against Orin NX. In comparison, GScore can barely achieve $21.6\times$ speedup. Our result also shows that coarse-grained filtering affects the performance of STREAMINGGS significantly. Without coarse-grained filtering, the speedup drops from $45.7\times$ to $22.2\times$, while VQ has a minimal impact on the performance. This is because VQ only affects the voxel streaming which is overlapped by the subsequent stages.

On energy saving, STREAMINGGS achieves $62.9\times$ energy reduction over Orin NX and $2.3\times$ energy reduction over GScore. The primary energy savings are from DRAM traffic reduction and efficient early-stage irrelevant Gaussian filtering. In addition, coarse-grained filtering and VQ contribute to $35.6\times$ and $5.8\times$ energy savings, respectively.

D. Sensitivity Study

Fig. 12 shows the sensitivity of rendering quality and energy savings to voxel size on one real-world scene, *train*. All variants are retrained according to our training procedure in Sec. V-A. Generally, smaller voxel sizes result in more cross-boundary Gaussians, leading to incorrect depth ordering. Our result shows that the rendering quality drops from 22.3 dB to

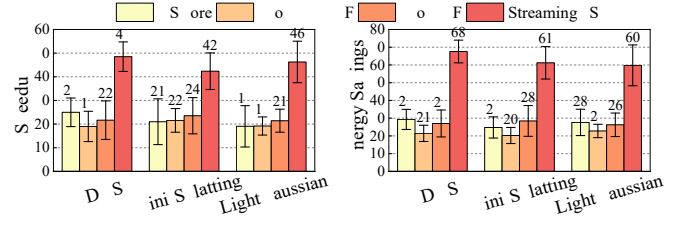


Fig. 11. End-to-end speedup and energy savings of our variants over the GPU baseline. The numbers are averaged over four datasets and normalized to the GPU baseline.

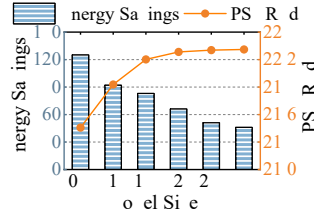


Fig. 12. The sensitivity of energy efficiency and rendering quality on the original 3DGS to voxel size. We only evaluate on *train* scene.

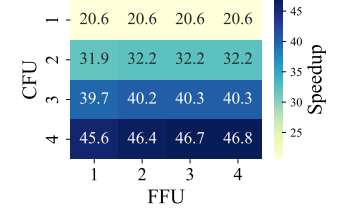


Fig. 13. The sensitivity of speedup on the original 3DGS to the numbers of CFUs and FFUs. Only *train* scene is evaluated.

21.5 dB when the voxel size decreases from 2 to 0.5. However, further increasing voxel size shows minimal impacts on rendering quality. Meanwhile, larger voxels might potentially include more irrelevant Gaussians in every voxel, increasing the filtering workload and lowering energy efficiency. We empirically find a voxel size of 2 has a good balance between performance and rendering quality.

Fig. 13 illustrates the sensitivity of performance to the numbers of coarse-grained filter units (CFUs) and fine-grained filter units (FFUs) in HFU (Sec. IV-C). We choose the same *train* scene and normalize all performance numbers to the GPU baseline. The results show that adding FFUs beyond the number of CFUs does not yield any speedup. In contrast, increasing the number of CFUs consistently boosts speedup, underscoring their importance in accelerating the filtering process. However, an excessive number of CFUs would lead to increased area overhead. Thus, we choose 4 CFUs with 1 FFU as our default configuration.

VI. CONCLUSION

This paper introduces STREAMINGGS that rethinks the 3DGS pipeline and proposes a new computing paradigm to process Gaussian points. Our fully streaming pipeline eliminates intermediate off-chip memory traffic and regularizes the off-chip accesses, all with a minimal but principle architecture augmentation. Overall, our algorithm-architecture co-design demonstrates $45.7\times$ and $2.1\times$ speedup compared to mobile GPUs and the state-of-the-art 3DGS accelerator, respectively.

VII. ACKNOWLEDGMENT

This work is sponsored by the National Natural Science Foundation of China (62472273, 62232015, 62402312).

REFERENCES

- [1] M. Pharr, W. Jakob, and G. Humphreys, *Physically based rendering: From theory to implementation*. MIT Press, 2023.
- [2] Y. Deng, Y. Ni, Z. Li, S. Mu, and W. Zhang, "Toward real-time ray tracing: A survey on hardware acceleration and microarchitecture techniques," *ACM Computing Surveys (CSUR)*, vol. 50, no. 4, pp. 1–41, 2017.
- [3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [4] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. P. Srinivasan, "Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5855–5864, 2021.
- [5] T. Wu, Y.-J. Yuan, L.-X. Zhang, J. Yang, Y.-P. Cao, L.-Q. Yan, and L. Gao, "Recent advances in 3d gaussian splatting," *arXiv preprint arXiv:2403.11134*, 2024.
- [6] G. Chen and W. Wang, "A survey on 3d gaussian splatting," *arXiv preprint arXiv:2401.03890*, 2024.
- [7] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics (ToG)*, vol. 42, no. 4, pp. 1–14, 2023.
- [8] G. Fang and B. Wang, "Mini-splatting: Representing scenes with a constrained number of gaussians," *arXiv preprint arXiv:2403.14166*, 2024.
- [9] J. C. Lee, D. Rho, X. Sun, J. H. Ko, and E. Park, "Compact 3d gaussian representation for radiance field," *arXiv preprint arXiv:2311.13681*, 2023.
- [10] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, and Z. Wang, "Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps," *arXiv preprint arXiv:2311.17245*, 2023.
- [11] W. Lin, Y. Feng, and Y. Zhu, "Metasapiens: Real-time neural rendering with efficiency-aware pruning and accelerated foveated rendering," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pp. 669–682, 2025.
- [12] X. Huang, H. Zhu, Z. Liu, W. Lin, X. Liu, Z. He, J. Leng, M. Guo, and Y. Feng, "Seele: A unified acceleration framework for real-time gaussian splatting," *arXiv preprint arXiv:2503.05168*, 2025.
- [13] "Meta Quest Pro specs."
- [14] "Apple Vision Pro screen refresh rate is up to 100Hz."
- [15] J. Wang, R. Shi, W. Zheng, W. Xie, D. Kao, and H.-N. Liang, "Effect of frame rate on user experience, performance, and simulator sickness in virtual reality," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 5, pp. 2478–2488, 2023.
- [16] "Nvidia jetson orin nx."
- [17] "Qualcomm QCS8550/QCM8550 Processors."
- [18] J. Lee, S. Lee, J. Lee, J. Park, and J. Sim, "Gscore: Efficient radiance field rendering via architectural support for 3d gaussian splatting," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pp. 497–511, 2024.
- [19] Y. Feng, W. Lin, Z. Liu, J. Leng, M. Guo, H. Zhao, X. Hou, J. Zhao, and Y. Zhu, "Potamoi: Accelerating neural rendering via a unified streaming architecture," *ACM Transactions on Architecture and Code Optimization*, 2024.
- [20] H. He, G. Li, F. Liu, L. Jiang, X. Liang, and Z. Song, "Gsarch: Breaking memory barriers in 3d gaussian splatting training via architectural support," in *2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2025.
- [21] Y. Feng, Z. Liu, J. Leng, M. Guo, and Y. Zhu, "Cicero: Addressing algorithmic and architectural bottlenecks in neural rendering by radiance warping and memory optimizations," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 1293–1308, IEEE, 2024.
- [22] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.
- [23] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.
- [24] "Micron 178-Ball, Single-Channel Mobile LPDDR3 SDRAM Features."
- [25] "Micron System Power Calculators."
- [26] S. Sarangi and B. Baas, "Deepscaletool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, IEEE, 2021.
- [27] L. Liu, J. Gu, K. Zaw Lin, T.-S. Chua, and C. Theobalt, "Neural sparse voxel fields," *Advances in Neural Information Processing Systems*, vol. 33, pp. 15651–15663, 2020.
- [28] A. Knapitsch, J. Park, Q.-Y. Zhou, and V. Koltun, "Tanks and temples: Benchmarking large-scale scene reconstruction," *ACM Transactions on Graphics (ToG)*, vol. 36, no. 4, pp. 1–13, 2017.
- [29] P. Hedman, J. Philip, T. Price, J.-M. Frahm, G. Drettakis, and G. Brostow, "Deep blending for free-viewpoint image-based rendering," *ACM Transactions on Graphics (ToG)*, vol. 37, no. 6, pp. 1–15, 2018.